

**AICTE - EDUSKILLS VIRTUAL INTERNSHIP
PROJECT REPORT**

"JSON Database Simulator"

SUBMITTED BY:

Student Name: Girik Goswami
Program: B.Tech
Branch: B.Tech(Computer Science and Engineering)

SUBMITTED TO:

IILM University, Greater Noida

Internship Details:

AICTE Student ID: STU69ecc320b22ec1777124128
APAAR ID: 317174378360
Internship Duration: 8 Weeks (Jun 2026 - Jul 2026)
Supported by: Eduskills Academy

Under the Guidance of

Project Manager
Eduskills

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms. Girik Goswami**, a student of **IILM University, Greater Noida**, has successfully completed **8-Week (Jun 2026 - Jul 2026)** AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled "**JSON Database Simulator**". His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

Sagarabala Behera

Mentor, EduSkills Academy

Date: 29-08-2026

DECLARATION

I hereby declare that the work presented in this Online Virtual Internship Project Report is my original work carried out under the guidance of my industry mentor and faculty guide. This work has not been submitted earlier for the award of any degree or diploma.

Girik Goswami

(Signature)

Date: 29-08-2026

ACKNOWLEDGEMENT

I express my sincere gratitude to **AICTE & EduSkills** for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor **Mr./Ms. Sagarabala Behera** for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of **B.Tech(Computer Science and Engineering)** for their encouragement and support.

ABSTRACT

This project details the construction of a lightweight, in-memory NoSQL database simulator. The system processes user commands through a text interface, supporting core CRUD operations (Create, Read, Update, Delete) and sophisticated querying, returning responses in JSON format. The aim is to provide a foundational understanding of database principles and data manipulation in a simplified environment.

Table of Contents

1. Introduction	2
2. Organization Profile	3
3. Internship Overview	4
4. Problem Statement	5
5. Methodology	6
6. Tools & Technologies	7
7. Implementation	8
8. Learning Outcomes	11
9. Challenges & Solutions	12
10. Conclusion	13
11. Future Scope	14
12. References	15
13. Annexure	16

1. Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **Python Full Stack Development With Project** and its application in modern software solutions.

2. Organization Profile

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

3. Internship Overview

- Internship Mode: **Online / Virtual**
- Duration: **8 Weeks**
- Platform Used: **EduSkills LMS**
- Role: **Intern**

4. Problem Statement & Objectives

Problem Statement:

The JSON Database Simulator is a project focused on building a functional, in-memory NoSQL database. This system is designed to emulate the behavior of a NoSQL database by processing various commands, including data creation, retrieval, modification, and deletion. It utilizes a text-based interface for user interaction and consistently returns results in the standardized JSON format. The simulator also incorporates advanced features for data filtering and generating snapshots, offering a comprehensive yet simplified environment for learning database management principles.

Objectives:

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

5. Project Description & Methodology

The development of the JSON Database Simulator was approached systematically. Initially, the project established an in-memory data structure using Python dictionaries to represent the database, with a specific dictionary key for each named database. Data within each database was stored as a list of JSON objects. The subsequent phases involved implementing the core CRUD operations: inserting new records by parsing JSON strings and appending them to the appropriate list; updating existing records by iterating through the data and modifying specific fields based on criteria; and deleting records by filtering out elements that match a given condition. Advanced functionalities such as filtering and creating data snapshots were realized using list comprehensions and lambda functions for efficient data retrieval and manipulation. The system was designed to accept commands via a text interface and output results as formatted JSON.

6. Tools & Technologies Used

Programming Language: Python

Libraries: json (for JSON parsing and serialization)

Data Structures: Python Dictionaries, Lists

Concepts: In-memory databases, NoSQL principles, CRUD operations, JSON data format, Text-based interfaces, Data filtering, Data snapshots.

7. Implementation & Results

Problem 1: Database Initialization

Description: The initial phase of the project involves setting up the fundamental data structure for the in-memory database. This requires defining how the database will be represented in memory, typically using Python dictionaries, and how individual named databases or collections will be organized within this structure, often as lists of records.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

# Database state
storage = {}
results = []

for line in lines:
    if not line.strip():
        result = {"status": "empty"}
    else:
        parts = line.split()
        command = parts[0]

        if command == "CREATE":
            name = parts[1]
            storage[name] = []
            result = {"status": "ok", "db": name}
        else:
            result = {"status": "error", "message": f"Unknown command: {command}"}

    results.append(result)

print(json.dumps(results))
```

Problem 2: Inserting Records

Description: A core requirement is the ability to add new data entries into the database. This involves parsing incoming data, which is expected in JSON string format, into a structured object that can be stored within the database's internal representation, typically a list of records.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

# Database state
storage = {}
records = []
results = []

for line in lines:
    if not line.strip():
        result = {"status": "empty"}
    else:
        parts = line.split(maxsplit=1)
        command = parts[0]

        if command == "CREATE":
            name = parts[1]
            storage[name] = []
            result = {"status": "ok", "db": name}
        elif command == "INSERT":
            json_str = parts[1]
            record = json.loads(json_str)
            records.append(record)
            result = {"status": "inserted", "id": record["id"]}
        else:
            result = {"status": "error", "message": f"Unknown command: {command}"}

    results.append(result)

print(json.dumps(results))
```

Problem 3: Updating Records

Description: The project must provide a mechanism to modify existing records within the database. This entails identifying the specific record(s) to be updated, often based on a unique identifier or other criteria, and then altering their associated values while maintaining data integrity.

```
import sys, json

data = sys.stdin.read()
if data.endswith('\n'):
    data = data[:-1]

db = []
results = []

for line in data.split('\n'):
    if not line.strip():
        results.append({"status": "skip"})
        continue

    parts = line.split(' ', 2)
    cmd = parts[0]

    if cmd == "INSERT":
        record = json.loads(parts[1])
        db.append(record)
        results.append({"status": "inserted", "id": record["id"]})

    elif cmd == "UPDATE":
        rid = int(parts[1])
        updates = json.loads(parts[2])
        match = None
        for item in db:
            if item["id"] == rid:
                match = item
                break
        if match:
            match.update(updates)
            results.append({"status": "updated", "id": rid})
        else:
            results.append({"status": "not_found", "id": rid})

    else:
        results.append({"status": "error", "message": "Unknown command"})

print(json.dumps(results))
```

Problem 4: Deleting Records

Description: Functionality for removing data from the database is essential. This process involves identifying records that meet a specified deletion criterion and then excluding them from the database's active data set.

```
import sys, json

data = sys.stdin.read()
if data.endswith('\n'):
    data = data[:-1]

db = []
results = []

for line in data.split('\n'):
    if not line.strip():
        results.append({"status": "skip"})
        continue

    parts = line.split(' ', 2)
    cmd = parts[0]

    if cmd == "INSERT":
        record = json.loads(parts[1])
        db.append(record)
        results.append({"status": "inserted", "id": record["id"]})

    elif cmd == "DELETE":
        rid = int(parts[1])
        before = len(db)
        db = [x for x in db if x["id"] != rid]
        if len(db) < before:
            results.append({"status": "deleted", "id": rid})
        else:
            results.append({"status": "not_found", "id": rid})

    else:
        results.append({"status": "error", "message": "Unknown command"})

print(json.dumps(results))
```

Problem 5: Filtering and Snapshot

Description: To enable more sophisticated data management, the simulator needs to support filtering capabilities, allowing users to retrieve subsets of data based on specific conditions. Additionally, the ability to create a 'snapshot' of the current database state or a filtered view is required, preserving the data at a particular point in time.

```
import sys, json

data = sys.stdin.read()
if data.endswith('\n'):
    data = data[:-1]

db = []
results = []

for line in data.split('\n'):
    if not line.strip():
        results.append({"status": "skip"})
        continue

    parts = line.split(' ', 2)
    cmd = parts[0]

    if cmd == "INSERT":
        record = json.loads(parts[1])
        db.append(record)
        results.append({"status": "inserted", "id": record["id"]})

    elif cmd == "QUERY":
        key = parts[1]
        value = parts[2] if len(parts) > 2 else ""
        matches = [x for x in db if str(x.get(key)) == value]
        results.append({"results": matches})

    elif cmd == "SNAPSHOT":
        results.append({"data": db})

    else:
        results.append({"status": "error", "message": "Unknown command"})

print(json.dumps(results))
```

8. Learning Outcomes

- Gained practical experience in designing and implementing an in-memory database system.
- Developed proficiency in handling JSON data, including parsing and serialization.
- Enhanced understanding of core database operations (CRUD) and their programmatic implementation.
- Improved skills in data manipulation using Pythonic techniques like list comprehensions and lambda functions.

9. Challenges Faced & Solutions

Challenge: Efficiently updating or deleting records in a large in-memory list without significant performance degradation.

Solution: Implemented filtering mechanisms using list comprehensions, which are generally optimized in Python for such tasks. For updates, iterating and modifying in place or creating a new list with updated items were considered, with the choice depending on specific update patterns.

Challenge: Handling malformed or inconsistent JSON input during record insertion or updates.

Solution: Incorporated error handling using `try-except` blocks around `json.loads()` to gracefully manage invalid JSON strings, providing informative feedback to the user. Input validation was also considered for data fields.

Challenge: Ensuring that filtering and snapshot operations were both accurate and performant.

Solution: Leveraged Python's built-in capabilities, such as lambda functions and list comprehensions, which are well-suited for these types of data transformations, ensuring that results were correctly generated without excessive computational overhead.

10. Conclusion

The JSON Database Simulator successfully demonstrates the fundamental principles of database management within an in-memory, NoSQL context. By implementing core CRUD operations and advanced querying features, the project provides a robust yet accessible platform for understanding data persistence and manipulation. The system's ability to process commands and return JSON responses highlights the practical application of these concepts in software development.

11. Future Scope

- * Implement data persistence to disk, allowing the database state to be saved and loaded.
- * Introduce more complex query operators, such as logical AND/OR, range queries, and regular expression matching.
- * Add support for transactions and concurrency control to handle multiple simultaneous operations.
- * Develop a more sophisticated command-line interface (CLI) or a simple API for interaction.

12. References

- * Python Documentation: Official guide for the Python programming language.
- * JSON (JavaScript Object Notation): Overview of the JSON data format.
- * Real Python: Tutorials and articles on Python programming and data handling.

13. Annexure

- Internship Completion Certificate
- Offer Letter